

Lecture 19

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

September 8, 2026

Top Down Translation

- Use predictive parsing to implement L- attributed definitions

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$
$E \rightarrow T$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$
$E \rightarrow T$

$E \rightarrow TR$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$
$E \rightarrow T$

$E \rightarrow TR$
$R \rightarrow +Tprint(+)R$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$
$E \rightarrow T$

$E \rightarrow TR$
$R \rightarrow +Tprint(+)R$
$R \rightarrow \epsilon$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$
$E \rightarrow T$

  - Tricky case:

$E \rightarrow TR$
$R \rightarrow +Tprint(+)R$
$R \rightarrow \epsilon$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \to E + T \quad print(+)$
$E \to T$

$E \to TR$
$R \to +Tprint(+)R$
$R \to \epsilon$

  - Tricky case:
    $A \to A_1 Y \quad A.a = g(A_1.a.Y.y)$

Top Down Translation

- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \rightarrow E + T \quad print(+)$
$E \rightarrow T$

$E \rightarrow TR$
$R \rightarrow +Tprint(+)R$
$R \rightarrow \epsilon$

  - Tricky case:
    $A \rightarrow A_1 Y \quad A.a = g(A_1.a.Y.y)$
    $A \rightarrow X \quad A.a = f(X.x)$

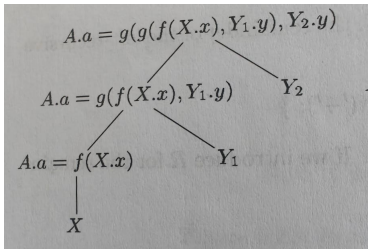
Top Down Translation

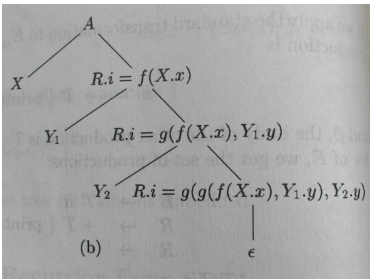
- Use predictive parsing to implement L- attributed definitions
- Eliminate Left Recursion
  - Easy Case:

$E \to E + T \quad print(+)$
$E \to T$

  - Tricky case:
    $A \to A_1 Y \quad A.a = g(A_1.a.Y.y)$
    $A \to X \quad A.a = f(X.x)$

$E \to TR$
$R \to +Tprint(+)R$
$R \to \epsilon$





Eliminating Left Recursion

$$A \rightarrow X \quad \{R.i = f(X.x)\} R \{A.a = R.s\}$$

Eliminating Left Recursion

$$A \to X \quad \{R.i = f(X.x)\} R \{A.a = R.s\}$$
$$R \to Y \{R_1 = g(R.i, Y.y\} R_1 \{R.s = R_1.s\}$$

Eliminating Left Recursion

$$A \to X \quad \{R.i = f(X.x)\}R\{A.a = R.s\}$$
$$R \to Y\{R_1 = g(R.i, Y.y\}R_1\{R.s = R_1.s\}$$
$$R \to \epsilon\{R.s = R.i\}$$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$E \rightarrow TR$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$E \rightarrow TR$
$R \rightarrow +T \quad print(+) \quad R$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$E \rightarrow TR$
$R \rightarrow +T \quad print(+) \quad R$
$R \rightarrow -T \quad print(-) \quad R$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$E \rightarrow TR$
$R \rightarrow +T \quad print(+) \quad R$
$R \rightarrow -T \quad print(-) \quad R$
$R \rightarrow \epsilon$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$E \rightarrow TR$
$R \rightarrow +T \quad print(+) \quad R$
$R \rightarrow -T \quad print(-) \quad R$
$R \rightarrow \epsilon$
$T \rightarrow num \quad print(num.val)$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$E \rightarrow TR$$

$E \rightarrow TR$
$R \rightarrow +T \quad print(+) \quad R$
$R \rightarrow -T \quad print(-) \quad R$
$R \rightarrow \epsilon$
$T \rightarrow num \quad print(num.val)$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$E \rightarrow TR$$
$$R \rightarrow +T \quad print(+) \quad R$$
$$R \rightarrow -T \quad print(-) \quad R$$
$$R \rightarrow \epsilon$$
$$T \rightarrow num \quad print(num.val)$$

$$E \rightarrow TR$$
$$R \rightarrow +TMR$$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$E \rightarrow TR$$
$$R \rightarrow +T \quad print(+) \quad R$$
$$R \rightarrow -T \quad print(-) \quad R$$
$$R \rightarrow \epsilon$$
$$T \rightarrow num \quad print(num.val)$$

$$E \rightarrow TR$$
$$R \rightarrow +TMR$$
$$R \rightarrow -TNR$$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$
\begin{aligned}
&E \rightarrow TR \\
&R \rightarrow +T \quad print(+) \quad R \\
&R \rightarrow -T \quad print(-) \quad R \\
&R \rightarrow \epsilon \\
&T \rightarrow num \quad print(num.val)
\end{aligned}
$$

$$
\begin{aligned}
&E \rightarrow TR \\
&R \rightarrow +TMR \\
&R \rightarrow -TNR \\
&R \rightarrow \epsilon
\end{aligned}
$$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$E \rightarrow TR$$
$$R \rightarrow +T \quad print(+) \quad R$$
$$R \rightarrow -T \quad print(-) \quad R$$
$$R \rightarrow \epsilon$$
$$T \rightarrow num \quad print(num.val)$$

$$E \rightarrow TR$$
$$R \rightarrow +TMR$$
$$R \rightarrow -TNR$$
$$R \rightarrow \epsilon$$
$$T \rightarrow num \quad \{print(num.val)\}$$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$E \rightarrow TR$$
$$R \rightarrow +T \quad print(+) \quad R$$
$$R \rightarrow -T \quad print(-) \quad R$$
$$R \rightarrow \epsilon$$
$$T \rightarrow num \quad print(num.val)$$

$$E \rightarrow TR$$
$$R \rightarrow +TMR$$
$$R \rightarrow -TNR$$
$$R \rightarrow \epsilon$$
$$T \rightarrow num \quad \{print(num.val)\}$$
$$M \rightarrow \epsilon \quad \{print(+)\}$$

Bottom up evaluation of inherited attributes

- Remove embedded actions from translation scheme
- Make transformation so that embedded actions occur only at the ends of their productions
- Replace each action by a distinct marker non terminal M and attach action at end of $M \rightarrow \epsilon$

$$
\begin{aligned}
&E \rightarrow TR \\
&R \rightarrow +T \quad print(+) \quad R \\
&R \rightarrow -T \quad print(-) \quad R \\
&R \rightarrow \epsilon \\
&T \rightarrow num \quad print(num.val)
\end{aligned}
$$

$$
\begin{aligned}
&E \rightarrow TR \\
&R \rightarrow +TMR \\
&R \rightarrow -TNR \\
&R \rightarrow \epsilon \\
&T \rightarrow num \quad \{print(num.val)\} \\
&M \rightarrow \epsilon \quad \{print(+)\} \\
&N \rightarrow \epsilon \quad \{print(-)\}
\end{aligned}
$$

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$
- Example *real* p, q, r

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$
- Example *real* p, q, r

$D \rightarrow T \quad \{L.in = T.type\} \quad L$

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$
- Example *real* p, q, r

$D \rightarrow T \quad \{L.in = T.type\} \quad L$
$T \rightarrow int \quad \{T.type = integer\}$

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$
- Example *real* p, q, r

$D \rightarrow T \quad \{L.in = T.type\} \quad L$
$T \rightarrow int \quad \{T.type = integer\}$
$T \rightarrow real \quad \{T.type = real\}$

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$
- Example *real* p, q, r

$D \rightarrow T \quad \{L.in = T.type\} \quad L$
$T \rightarrow int \quad \{T.type = integer\}$
$T \rightarrow real \quad \{T.type = real\}$
$L \rightarrow \{L_1.in = L.in\} \quad L_1, id \quad \{addtype(id.entry, L.in)\}$

Inheriting attribute on parser stacks

- Bottom up parser reduces rhs of $A \rightarrow XY$ by removing XY from stack and putting A on the stack
- Synthesized attributes of X_s can be inherited by Y by using the copy rule $Y.i = X.s$
- Example *real* p, q, r

$D \rightarrow T \quad \{L.in = T.type\} \quad L$
$T \rightarrow int \quad \{T.type = integer\}$
$T \rightarrow real \quad \{T.type = real\}$
$L \rightarrow \{L_1.in = L.in\} \quad L_1, id \quad \{addtype(id.entry, L.in)\}$
$L \rightarrow id \quad \{addtype(id.entry, L.in)\}$

Analysis

State stack	INPUT	PRODUCTION
	real p,q,r	
real	p,q,r	
T	p,q,r	T → real
Tp	,q,r	
TL	,q,r	L → id
TL,	q,r	
TL,q	,r	
TL	,r	L → L,id
TL,	r	
TL,r	-	
TL	-	L → L,id
D	-	D →TL

Analysis

- Every time a reduction to L is made value of T type is just below it

Analysis

- Every time a reduction to L is made value of T type is just below it
- Use the fact that $T.val$ (type information) is at a known place in the stack

Analysis

- Every time a reduction to L is made value of T type is just below it
- Use the fact that $T.val$ (type information) is at a known place in the stack
- When production $L \rightarrow id$ is applied, $id.entry$ is at the top of the stack and $T.type$ is just below it, therefore,

Analysis

- Every time a reduction to L is made value of T type is just below it
- Use the fact that $T.val$ (type information) is at a known place in the stack
- When production $L \rightarrow id$ is applied, $id.entry$ is at the top of the stack and $T.type$ is just below it, therefore,
  $addtype(id.entry, L.in) \longleftrightarrow addtype(val[top], val[top-1])$

Analysis

- Every time a reduction to L is made value of T type is just below it
- Use the fact that $T.val$ (type information) is at a known place in the stack
- When production $L \rightarrow id$ is applied, $id.entry$ is at the top of the stack and $T.type$ is just below it, therefore,
  $addtype(id.entry, L.in) \longleftrightarrow addtype(val[top], val[top-1])$
- Similarly when production $L \rightarrow L_1, id$ is applied $id.entry$ is at the top of the stack and $T.type$ is three places below it, therefore,

Analysis

- Every time a reduction to L is made value of T type is just below it
- Use the fact that $T.val$ (type information) is at a known place in the stack
- When production $L \rightarrow id$ is applied, $id.entry$ is at the top of the stack and $T.type$ is just below it, therefore,
  $addtype(id.entry, L.in) \longleftrightarrow addtype(val[top], val[top-1])$
- Similarly when production $L \rightarrow L_1, id$ is applied $id.entry$ is at the top of the stack and $T.type$ is three places below it, therefore,
  $addtype(id.entry, L.in) \longleftrightarrow addtype(val[top], val[top-3])$

Analysis

$D \rightarrow TL$
$T \rightarrow int \quad val[top] = integer$
$T \rightarrow real \quad val[top] = real$
$L \rightarrow L, id \quad addtype(val[top], val[top-3])$
$L \rightarrow id \quad addtype(val[top], val[top-1])$